

Multiple Choice Questions:

1. Which C# operator is used to obtain type information at compile time?
 - A) `GetType()`
 - B) `typeof`
 - C) `TypeOf()`
 - D) `GetClass()`
2. What method is used to dynamically load an external assembly in C#?
 - A) `Assembly.Create()`
 - B) `Assembly.Run()`
 - C) `Assembly.LoadFrom()`
 - D) `Assembly.Execute()`
3. Which class is used to represent information about methods in reflection?
 - A) `FieldInfo`
 - B) `ConstructorInfo`
 - C) `MethodInfo`
 - D) `PropertyInfo`
4. The `GetType()` method is a member of which base class in C#?
 - A) `Type`
 - B) `Assembly`
 - C) `Object`
 - D) `Attribute`
5. What is the purpose of the `[StorableClass]` attribute in the code example?
 - A) To encrypt the class
 - B) To define serializable fields
 - C) To indicate that the class should be saved
 - D) To define class visibility
6. What is checked first in the `SaveUtils.Save` method before saving any object?
 - A) The number of fields
 - B) If the object is null
 - C) If `Storable` fields are public
 - D) If the class has the `StorableClassAttribute`
7. Which binding flag is used to include both public and private fields?
 - A) `BindingFlags.Static`
 - B) `BindingFlags.Instance`
 - C) `BindingFlags.All`
 - D) `BindingFlags.Public | BindingFlags.NonPublic`
8. In the example, what does the `SaveUtils.Save()` method do instead of actual saving?

- A) Encrypts the object
 - B) Prints the save parameters to the console
 - C) Stores it in a database
 - D) Sends it over the network
9. What does the `StorableAttribute` constructor require as a parameter?
- A) Field type
 - B) Attribute target
 - C) Field name as string
 - D) Object reference
10. Which of the following types is NOT derived from `MemberInfo`?
- A) `FieldInfo`
 - B) `MethodInfo`
 - C) `Type`
 - D) `ConstructorInfo`

Answer Key:

1. B) **typeof**
2. C) **Assembly.LoadFrom()**
3. C) **MethodInfo**
4. C) **Object**
5. C) To indicate that the class should be saved
6. D) If the class has the **StorableClassAttribute**
7. D) **BindingFlags.Public | BindingFlags.NonPublic**
8. B) Prints the save parameters to the console
9. C) Field name as string
10. C) **Type**

Multiple-Answer Questions:

1. Which of the following are types derived from `MemberInfo`?
 - ☐ `FieldInfo`
 - ☐ `MethodBase`
 - ☐ `Assembly`
 - ☐ `ConstructorInfo`
2. Which can be used to retrieve a type in C#?
 - ☐ `typeof`
 - ☐ `GetType()`
 - ☐ `GetFields`
 - ☐ `LoadFrom`
3. What methods can be used to obtain metadata about fields in a type?
 - ☐ `type.GetFields()`
 - ☐ `type.GetType()`
 - ☐ `type.GetMethods()`
 - ☐ `type.GetCustomAttributes()`
4. Which attributes are used in the provided `Student` class example?
 - ☐ `[Serializable]`
 - ☐ `[Storable]`
 - ☐ `[Obsolete]`
 - ☐ `[StorableClass]`
5. What does the `BindingFlags` enumeration control in reflection?
 - ☐ Accessibility of fields
 - ☐ Whether static or instance members are retrieved
 - ☐ The name of the class
 - ☐ What metadata is saved to disk
6. In the `SaveUtils.Save` method, which actions occur?
 - ☐ Object type is retrieved using reflection
 - ☐ All public fields are serialized
 - ☐ Console output is used instead of real saving
 - ☐ Fields are ignored unless they have specific attributes
7. Which are valid targets for a custom attribute defined using `[AttributeUsage]`?
 - ☐ Classes
 - ☐ Assemblies
 - ☐ Fields
 - ☐ Variables
8. What are purposes of using custom attributes in C# reflection?

- ☐ Adding metadata to classes or fields
 - ☐ Changing the class name at runtime
 - ☐ Controlling behavior during serialization
 - ☐ Preventing a class from compiling
9. Which components are involved in loading an assembly and inspecting its types?
- ☐ `Assembly.LoadFrom`
 - ☐ `typeof`
 - ☐ `GetTypes()`
 - ☐ `new Operator`
10. In the context of the document, what are valid uses of the reflection API?
- ☐ Inspecting method names at runtime
 - ☐ Dynamically changing class definitions
 - ☐ Extracting field values of an object
 - ☐ Assigning new attributes during execution

Answer Key:

1. **`FieldInfo`, `MethodBase`, `ConstructorInfo`**
2. **`typeof`, `GetType()`**
3. **`type.GetFields()`**
4. **`[Storable]`, `[StorableClass]`**
5. **Accessibility of fields, Whether static or instance members are retrieved**
6. **Object type is retrieved using reflection, Console output is used instead of real saving, Fields are ignored unless they have specific attributes**
7. **Classes, Fields**
8. **Adding metadata to classes or fields, Controlling behavior during serialization**
9. **`Assembly.LoadFrom`, `GetTypes()`**
10. **Inspecting method names at runtime, Extracting field values of an object**

True or False Questions:

1. The `typeof` operator is evaluated at runtime in C#.
2. `FieldInfo` is used to access method details in a class.
3. `Assembly.LoadFrom("MyLib.dll")` is used to load an assembly dynamically.
4. `GetType()` is a method inherited from the `Object` class.
5. The `StorableClassAttribute` can be applied to both classes and fields.
6. In the `SaveUtils.Save` method, only fields marked with the `Storable` attribute are processed.
7. `MethodInfo` provides information about the constructors of a class.
8. The `BindingFlags.Instance` flag is used to retrieve static fields only.
9. The `Student` class in the example is marked with a custom attribute to indicate it should be saved.
10. The reflection API in C# allows examining metadata about types at runtime.

Answer Key:

1. **False**
2. **False**
3. **True**
4. **True**
5. **False**
6. **True**
7. **False**
8. **False**
9. **True**
10. **True**

Complete the Sentence:

1. The `GetType` method is defined in the _____ class.
2. The `C#` operator used to get type information at compile time is _____.
3. To retrieve all the types from a loaded assembly, we use the method _____.
4. The class that provides information about fields is called _____.
5. The custom attribute that marks a class as savable is _____.
6. The custom attribute applied to a field in the `Student` class is _____.
7. The method that saves an object in `SaveUtils` is named _____.
8. The method used to retrieve custom attributes from a type is _____.
9. The attribute target for `StorableAttribute` is defined using _____.
10. The binding flags used to retrieve fields include the flag named _____.

Answer Key:

1. **Object**
2. **typeof**
3. **GetTypes**
4. **FieldInfo**
5. **StorableClass**
6. **Storable**
7. **Save**
8. **GetCustomAttributes**
9. **AttributeUsage**
10. **Instance**